

CampusCart

Project Explanation, Workflow, Interview Questions and Answers

Prepared for student interview/viva revision

CampusCart is a full-stack web application for hostel students. It helps students combine food or grocery orders from platforms like Blinkit, Zepto, Instamart, Swiggy, and Zomato so they can cross free-delivery limits and save money.

Best 20-second explanation:

CampusCart helps hostel students combine food and grocery delivery orders to save delivery charges. It uses a Next.js frontend, Express backend, and MongoDB database. The backend authenticates users with JWT and matches orders using platform, hostel, amount, and timing.

1. Project Overview

- Problem: Students often place small delivery orders and pay extra delivery charges.
- Solution: CampusCart finds nearby students ordering from the same platform and helps them combine orders.
- Target users: College and hostel students.
- Main result: Students save delivery fees and coordinate orders more easily.

Example:

Student A creates a Blinkit order of Rs 220. Student B from the same hostel creates a Blinkit order of Rs 190. Together they reach Rs 410, which may cross the free-delivery threshold. CampusCart suggests them as a match.

2. Tech Stack

Layer	Technologies	Purpose
Frontend	Next.js, React, TypeScript, Tailwind CSS, Radix UI, Lucide Icons	Build Pages, UI components, forms, dashboard, active orders, and chat
Backend	Node.js, Express.js, TypeScript, JWT, bcryptjs, express-rate-limit, passport, nodemailer	Create APIs, authenticate users, validate data, handle order and match
Database	MongoDB, Mongoose	Store users, colleges, hostels, orders, rooms, participants, messages,
Realtime planned	Socket.IO	Useful for live chat and room updates.

Why These Technologies Were Used

Why Next.js?

Next.js gives file-based routing, good project structure, performance benefits, and easier deployment compared to plain React.

Why React?

React helps build reusable UI components such as cards, forms, navbar, sidebar, order cards, and dashboard widgets.

Why TypeScript?

TypeScript catches mistakes early by defining types for users, orders, messages, platforms, and API responses.

Why Tailwind CSS?

Tailwind allows fast and consistent styling using utility classes.

Why Express.js?

Express is simple, flexible, and widely used for building REST APIs.

Why MongoDB?

MongoDB stores flexible JSON-like documents, which fits users, orders, rooms, chats, and notifications well.

Why Mongoose?

Mongoose gives structure to MongoDB by defining schemas and models.

Why JWT?

JWT lets the frontend prove that a user is logged in when calling protected backend APIs.

3. Project Workflow

Main user flow: User opens CampusCart -> logs in or registers -> selects college and hostel -> creates an order -> system searches for similar active orders -> matching users are suggested -> users create or join a room -> users coordinate through chat -> one combined order is placed -> delivery fee is saved.

Backend request flow:

- Frontend sends an API request.
- Express route receives the request.
- Middleware checks authentication and validates input.
- Controller handles request and response.
- Service applies business logic.
- Mongoose model reads or writes MongoDB data.
- Backend returns JSON response to frontend.

Authentication flow:

- User enters email and password.
- Backend checks if the user exists.
- bcrypt compares the entered password with the hashed password.
- If valid, backend creates a JWT token.
- Frontend stores the token.
- Frontend sends the token for protected requests.

Order matching flow:

- User creates an order with platform, amount, hostel, note, and time window.
- Backend stores the order as open.
- Backend searches active orders from the same platform and hostel.
- Backend checks amount similarity and time window.
- Backend returns match suggestions with scores and reasons.

4. Current Status And Honest Explanation

- The project has a complete frontend UI and backend API structure.
- Earlier, frontend pages used mock data. The correct final flow is frontend -> backend API -> MongoDB.
- Payment is not needed for the current explanation.
- The matching algorithm is currently rule-based, not machine-learning based.
- Socket.IO is included and can be used later for realtime chat.

5. Interview Questions And Answers

Project Concept

Why did you choose this project idea?

I chose this idea because students often order food or groceries separately and pay extra delivery charges. CampusCart solves a real student-life problem by helping hostel students combine orders and save money.

What real-world problem does CampusCart solve?

It solves the problem of unnecessary delivery charges on small orders by grouping students who are ordering from the same platform around the same time.

Who are the target users?

College students, especially hostel students, who frequently order food or groceries online.

How is this different from normal food delivery apps?

Food delivery apps deliver individual orders. CampusCart helps users coordinate and combine orders before placing them on those apps.

Why would students use your platform?

Because they can save delivery fees, find nearby students ordering from the same app, and coordinate easily.

What is the main value of CampusCart?

The main value is cost saving through order sharing and smarter matching between nearby students.

What is the scope of your project?

The scope includes user login, hostel/college selection, order creation, active order listing, matching, room creation, and chat structure.

What features are completed and planned?

Completed: UI, backend APIs, authentication, order models, matching logic, and database schemas. Planned: stronger realtime chat, payment integration, notifications, and deployment.

What assumptions did you make?

I assumed students from the same hostel can coordinate easily, orders are time-sensitive, and users are willing to combine orders if it saves delivery fees.

What is the biggest limitation?

The current matching is rule-based and features like production realtime chat can be improved further.

Frontend

Why Next.js instead of plain React?

Next.js gives file-based routing, better structure, performance benefits, and easier deployment compared to plain React.

What is the role of React?

React is used to build reusable UI components like cards, buttons, forms, navbar, order cards, and dashboard sections.

What is the role of TypeScript in frontend?

TypeScript helps catch errors early by defining data types for users, orders, platforms, and messages.

Why Tailwind CSS?

Tailwind allows fast styling using utility classes and helps maintain a consistent modern UI.

What are reusable components?

Reusable components are UI blocks used in multiple places, such as Button, Card, Navbar, Sidebar, and OrderCard.

What is the difference between a page and a component?

A page represents a full route or screen. A component is a smaller reusable UI part inside a page.

What is the purpose of the app folder?

In Next.js, the app folder defines routes and layouts using file-based routing.

How does routing work in your frontend?

Each folder inside app becomes a route. For example, login/page.tsx becomes /login.

What is a client component?

A client component runs in the browser and can use state, events, localStorage, and router navigation.

Why do some files use use client?

It is used when a component needs browser-side features like button clicks, forms, hooks, or navigation.

How do you protect dashboard pages?

Check whether a JWT token exists. If not, redirect the user to login.

How does frontend know the backend URL?

Through an environment variable such as NEXT_PUBLIC_API_URL.

What are the risks of storing JWT in localStorage?

If the site has an XSS attack, the token can be stolen. HTTP-only cookies are safer for production.

Backend

Why Node.js?

Node.js allows JavaScript or TypeScript to run on the server and works well for API-based applications.

Why Express.js?

Express is simple, flexible, and widely used for creating REST APIs.

What is an API?

An API is a way for frontend and backend to communicate.

What is a REST API?

A REST API uses HTTP methods like GET, POST, PUT, and DELETE to work with resources.

What are the main API endpoints?

Auth APIs, order APIs, user APIs, message APIs, college APIs, hostel APIs, and notification APIs.

What is the difference between route, controller, and service?

Route defines the URL, controller handles request and response, and service contains business logic.

Why separate controller and service?

It keeps code clean, organized, and easier to maintain.

What middleware did you use?

Authentication middleware, validation middleware, error handler, CORS, and JSON body parser.

What does CORS do?

CORS allows frontend and backend running on different ports to communicate.

Why do we need dotenv?

dotenv loads secret values like MongoDB URL and JWT secret from a .env file.

How do you prevent duplicate users?

Before registration, the backend checks if a user with the same email already exists.

How are passwords stored?

Passwords are hashed using bcrypt before being saved in MongoDB.

What happens if JWT expires?

The backend rejects protected requests and the user must login again.

Database

Why MongoDB?

MongoDB stores JSON-like documents and fits well with users, orders, rooms, and messages.

What is Mongoose?

Mongoose is an ODM that helps define schemas and interact with MongoDB using models.

What is a schema?

A schema defines the structure of a document.

What is a model?

A model is used to create, read, update, and delete documents from a collection.

What collections are used?

Users, Colleges, Hostels, Orders, Rooms, Participants, Messages, Matches, and Notifications.

How is a user linked to a hostel?

The user stores hostelId, which references the Hostel collection.

How is a hostel linked to a college?

The hostel stores collegelId, which references the College collection.

How is an order linked to a user?

The order stores ownerId, which references the User collection.

What is ObjectId?

ObjectId is MongoDB's unique ID used to identify documents.

What is populate in Mongoose?

populate replaces an ID with actual related document data, like owner name or hostel name.

What is a TTL index?

A TTL index automatically deletes documents after a certain expiry time.

Would SQL be better?

SQL could be better if relationships and transactions become very strict.

Authentication

Explain login flow.

User enters email and password, backend verifies password, creates JWT, and frontend stores the token.

Explain registration flow.

User submits details, backend checks duplicate email, hashes password, saves user, and returns token.

What is bcrypt?

bcrypt is a password hashing library.

What is salting?

Salting adds random data before hashing so the same passwords produce different hashes.

What is JWT?

JWT is a signed token used to prove user identity.

What data do you store inside JWT?

User ID and email.

Where does frontend send JWT?

In the Authorization header.

What does Bearer token mean?

It means the request is carrying a token for authentication.

How would you implement logout?

Remove the token from frontend storage.

How would you improve authentication security?

Use HTTP-only cookies, refresh tokens, a strong JWT secret, and rate limiting.

Order System

How does a user create an order?

The user fills the order form, frontend sends data to backend, and backend saves the order in MongoDB.

What fields are needed?

Platform, amount, hostelId, note, and orderWindow.

What is orderWindow?

The time duration for which the order remains open.

What is expiresAt?

The exact time when the order expires.

Why should orders expire?

Delivery orders are time-sensitive and should not stay active forever.

What are possible order statuses?

open, matching, confirmed, delivered, and cancelled.

How do you get active orders?

Backend filters orders with status open and expiry time greater than current time.

How do you filter orders by hostel?

Use hostelId in the query.

Why different free-delivery thresholds?

Each delivery platform has different free-delivery rules.

How would you prevent a user from joining twice?

Use a unique check with roomId and userId in the participant records.

Matching Algorithm

Explain your matching algorithm.

It finds active orders with the same platform, same hostel, similar amount, and valid time window.

Is your matching AI-based or rule-based?

Currently it is rule-based.

Why rule-based matching?

It is simple, explainable, and suitable for an initial version.

What factors are used?

Platform, hostel, amount similarity, and order expiry/time window.

What is a good match?

Same platform, same hostel, close amount, and active order.

What is a bad match?

Different platform, far location, expired order, or very different amount.

How does amount affect score?

A smaller amount difference gives a higher match score.

How does hostel affect score?

Same hostel is preferred because coordination is easier.

Can machine learning be added later?

Yes, with enough historical order data such as successful matches, timings, preferences, and ratings.

What happens if no match is found?

The system can keep the order open until expiry or show nearby active orders.

Rooms And Chat**What is a room?**

A room is a group space for students combining an order.

Why do you need rooms?

To coordinate participants, orders, and messages in one place.

When is a room created?

When matched users decide to group orders.

What data is stored in a room?

Hostel, platform, order IDs, expiry, and status.

What is Socket.IO?

A library for realtime communication between frontend and backend.

Why is Socket.IO useful?

Messages and updates appear instantly without refreshing.

HTTP vs WebSocket?

HTTP is request-response. WebSocket keeps a live connection open.

How are messages stored?

Messages are stored with roomId, senderId, content, and timestamp.

Why store messages?

So previous chat history can be loaded later.

How would you prevent unauthorized chat access?

Check if the user belongs to that room before returning messages.

Frontend-Backend Connection

How did you connect frontend and backend?

The frontend sends HTTP requests to backend API endpoints.

What is the backend base URL?

Usually `http://localhost:5000/api` in local development.

Why different ports?

Frontend and backend are separate servers during development.

What issue does CORS solve?

It allows frontend on port 3000 to call backend on port 5000.

How does frontend send data?

Using JSON request bodies in POST or PUT requests.

How does frontend receive response?

The backend sends JSON response data.

What is JSON?

A lightweight data format used between frontend and backend.

How did you replace mock data?

By stopping local mock imports and fetching real data from backend APIs.

What problems came while connecting?

CORS, missing token, wrong URL, data shape mismatch, and MongoDB connection issues.

How do you check backend is running?

Open the health-check endpoint.

Testing And Debugging

How did you debug backend errors?

Using terminal logs, API testing tools, and error responses.

How did you debug frontend errors?

Using browser console, network tab, and React error messages.

What does nodemon not recognized mean?

Dependencies were not installed. Running `npm install` fixes it.

What does MongoDB connection failed mean?

Wrong URI, MongoDB not running, or network/IP issue.

What does 401 Unauthorized mean?

Token is missing or invalid.

What does 404 Not Found mean?

The API route is wrong or missing.

What does 500 Internal Server Error mean?

There is an internal backend error.

How would you test APIs?

Use Postman or Thunder Client.

Which APIs are most important to test?

Login, register, create order, get active orders, and find matches.

Security

What security measures are used?

Password hashing, JWT authentication, validation, and environment variables.

How are passwords protected?

Using bcrypt hashing.

How are protected APIs secured?

Using JWT authentication middleware.

What is input validation?

Checking user data before processing it.

Why is validation important?

It prevents bad data and reduces security issues.

What is rate limiting?

Limiting repeated requests to prevent spam or brute-force attempts.

Why should .env not be uploaded?

It contains secrets like database URL and JWT secret.

How would you prevent fake users?

Use college email verification or OTP verification.

How would you protect admin routes?

Use role-based authorization.

Production security improvements?

Use HTTPS, HTTP-only cookies, rate limiting, secure CORS, logging, and validation.

Deployment

How would you deploy this project?

Frontend on Vercel, backend on Render or Railway, and database on MongoDB Atlas.

Where would you deploy frontend?

Vercel.

Where would you deploy backend?

Render, Railway, or Fly.io.

Where would you host MongoDB?

MongoDB Atlas.

What is MongoDB Atlas?

A cloud-hosted MongoDB service.

What environment variables are needed?

MONGODB_URI, JWT_SECRET, PORT, and frontend/backend URLs.

What is production mode?

An optimized version of the app for real users.

Frontend build command?

npm run build.

Backend build command?

npm run build.

How handle CORS in production?

Allow only the deployed frontend domain.

Project Improvement

What feature would you add next?

Realtime chat and better matching.

How would you make matching smarter?

Use trust score, order history, location, preferred timings, and ML later.

How would you add notifications?

Use database notifications and later push notifications.

How would you add admin panel?

Create admin routes and UI to manage users, colleges, reports, and orders.

How would you add order history?

Store completed orders and show them in profile.

How would you improve trust score?

Increase score for successful participation and reduce it for cancellations or misuse.

How would you scale this app?

Use indexes, pagination, caching, MongoDB Atlas, and optimized queries.

How would you reduce database load?

Cache active orders and paginate data.

How would you handle many chat messages?

Paginate messages and load older messages on scroll.

How would you improve UI/UX?

Add better loading states, empty states, mobile flow, and clearer order status.

6. Tough Questions And Strong Answers

Why not just use WhatsApp groups?

WhatsApp is unstructured. CampusCart gives structured order creation, matching, expiry, status tracking, user profiles, and future payment tracking.

Is this really AI?

Currently it uses rule-based intelligent matching. In the future, AI can be added to learn user behavior, ordering patterns, preferred timings, and trust score.

What was the hardest part?

Connecting frontend and backend properly, replacing mock data with real API data, handling authentication tokens, and keeping frontend data format consistent with backend response.

What is your project's limitation?

The current version focuses on order matching and coordination. Realtime chat, advanced matching, and production-level security can be improved later.

How will you scale it?

Use MongoDB Atlas, backend deployment on cloud, indexes for active orders, Redis caching, pagination for orders/messages, and Socket.IO rooms for realtime communication.

How do you make sure students actually pay?

Use trust score, payment status, ratings, and later a proper payment gateway or proof system.

What if one student cancels after joining?

Update participant or order status and reduce trust score if needed.

Who places the final order?

Usually the order owner or room creator places the final order.

What if no match is available?

The order remains open until expiry or the user can create a fresh order later.

What did you learn?

I learned full-stack development, REST APIs, authentication, MongoDB schemas, frontend-backend integration, TypeScript, and project architecture.

7. Problems You May Have Faced While Making This

Frontend-backend connection

Frontend and backend run on different ports, which can cause CORS or wrong API URL issues. The solution is to use cors in backend and a frontend environment variable for the API URL.

Mock data to real data

Frontend initially used static data. The solution is to replace local data imports with API calls.

Authentication token handling

Protected routes fail if token is missing. The solution is to store JWT after login and send it in the Authorization header.

MongoDB connection

Connection can fail due to wrong URI, local MongoDB not running, Atlas IP not whitelisted, or missing .env file.

nodemon not recognized

This happens when dependencies are not installed. Running npm install inside backend fixes it.

Route confusion

Frontend routes and backend API routes are different. Example: frontend page /active-orders uses backend API /api/orders.

Data shape mismatch

Frontend may expect creatorName while backend returns ownerId.name. The solution is to transform API response or update frontend types.

Duplicate server start

If app.ts and index.ts both start the server, the server may listen twice. Keep setup in app.ts and server start in index.ts.

Validation errors

Backend rejects missing or invalid fields like hostelId, amount, or platform. Match frontend form data with backend validation.

Realtime chat complexity

Normal HTTP does not update instantly. Socket.IO can be added later for live messages.

8. Final Answers To Memorize

Best 1-minute explanation:

CampusCart is a full-stack web application for hostel students to combine delivery orders and save delivery fees. The frontend is built with Next.js, React, TypeScript, Tailwind CSS, and Radix UI. The backend is built with Node.js, Express, TypeScript, JWT authentication, bcrypt password hashing, and MongoDB with Mongoose. The user logs in, selects their college and hostel, creates an order, and the backend matches that order with other active orders based on platform, hostel, amount, and time window. Once matched, users can join a room, coordinate the combined order, and save delivery charges. The project follows a layered architecture with routes, middleware, controllers, services, and database models.

Best 20-second explanation:

CampusCart helps hostel students combine food and grocery delivery orders to save delivery charges. It uses a Next.js frontend, Express backend, and MongoDB database. The backend authenticates users with JWT and matches orders using platform, hostel, amount, and timing.

Simple workflow to say in viva:

- User logs in.
- User selects college and hostel.
- User creates an order.
- Backend finds similar active orders.
- Users join a room and coordinate.
- Combined order is placed.
- Delivery charges are saved.